RAG系统执行顺序完整指南

📋 目录

1. 系统整体执行流程
2. 离线阶段：知识库构建
3. 在线阶段：问答流程
4. 各文件内部执行顺序
5. 完整调用关系图
6. 数据流向图

---

1. 系统整体执行流程

1.1 两大阶段

```
RAG系统运行
├── 离线阶段（初始化和知识库构建）
│    └── 只执行一次，或在更新知识库时执行
│
└── 在线阶段（问答流程）
     └── 每次用户提问都会执行
```

1.2 完整执行时间线

```
用户运行程序: streamlit run main.py
     ↓
【阶段1: 系统初始化】（只执行一次）
     ↓
加载配置 → 检查系统状态 → 初始化模型 → 加载/创建向量库 → 初始化检索器 → 创建问答链
     ↓
【阶段2: Web界面显示】（持续运行）
     ↓
显示界面 → 等待用户输入
     ↓
【阶段3: 问答循环】（每次提问执行一次）
     ↓
```

```
用户提问 → 问题改写 → 检索文档 → 格式化上下文 → 生成答案 → 显示结果
    ↓
等待下一次提问（循环）
```

---

2. 离线阶段：知识库构建

2.1 首次运行（没有向量库时）

```
main.py 执行
    ↓
【第1步】加载配置
    ├─ 读取 config.py
    └─ 获取所有配置参数
    ↓
【第2步】初始化系统组件
    ↓
    ├─ 2.1 初始化嵌入模型
    │   └─ knowledge/knowledge_base.py → init_embeddings()
    │       ├─ 导入 HuggingFaceEmbeddings
    │       ├─ 创建嵌入模型实例
    │       └─ 返回 embeddings 对象
    │
    ├─ 2.2 初始化大模型
    │   └─ generation/qa_chain.py → init_llm()
    │       ├─ 导入 Ollama
    │       ├─ 创建 Ollama 实例
    │       └─ 返回 llm 对象
    │
    ├─ 2.3 初始化向量库（首次运行）
    │   └─ knowledge/knowledge_base.py → init_vector_db(embeddings)
    │       ├─ 检查 Chroma 数据库路径是否存在
    │       ├─ 不存在 → 创建新数据库
    │       ├─ 创建空的向量库
    │       └─ 返回 vector_db 和空文档列表
    │
    ├─ 2.4 初始化BM25检索器（首次为空）
    │   └─ knowledge/knowledge_base.py → init_bm25_retriever(all_docs)
    │       ├─ 检查文档列表是否为空
    │       └─ 返回 None（因为还没有文档）
    │
    ├─ 2.5 初始化重排序器
    │   └─ retrieval/reranker.py → init_reranker()
```

```
    │        ├─ 导入 HuggingFaceCrossEncoder
    │        ├─ 创建交叉编码器模型
    │        └─ 返回 reranker 对象
    │
    └─ 2.6 构建最终检索器
        └─ retrieval/retriever.py → build_final_retriever(vector_db,
bm25_retriever, reranker)
             ├─ 创建向量检索器
             ├─ 判断是否使用混合检索（bm25为None，不使用）
             ├─ 判断是否使用重排序
             └─ 返回最终检索器
    ↓
【第3步】创建问答链
    └─ generation/qa_chain.py → QAChain(llm, final_retriever)
        └─ 返回 qa_chain 对象
    ↓
【第4步】显示Web界面
    └─ main.py → render_sidebar() + render_chat_interface()
    ↓
等待用户操作
```

2.2 用户点击"更新知识库"按钮

```
用户点击"🔄 更新知识库"按钮
    ↓
【第1步】扫描知识库文件夹
    └─ knowledge/knowledge_base.py →
build_or_update_knowledge_base(embeddings, vector_db, all_docs)
        ├─ 遍历 KNOWLEDGE_BASE_DIR 文件夹
        ├─ 收集所有支持的文件（.pdf, .docx, .md, .txt）
        └─ 得到文件列表
    ↓
【第2步】筛选需要更新的文件（增量更新）
    ├─ 提取已有文档的MD5值
    ├─ 计算新文件的MD5值
    └─ 筛选MD5不匹配的文件
    ↓
【第3步】批量处理文件
    ↓
对于每个待更新的文件:
    ├─ 3.1 加载文档
    │   └─ knowledge/document_loader.py →
DocumentLoader.load_document(file_path)
    │        ├─ 获取文件扩展名
```

```
    │        ├─ 根据扩展名选择加载器（PDF/Word/Markdown/Text）
    │        ├─ 调用加载器加载文档内容
    │        ├─ 计算文件MD5（utils/file_utils.py → get_file_md5）
    │        ├─ 获取文件名（utils/file_utils.py → get_file_name）
    │        ├─ 添加元数据（file_name, file_md5, update_time）
    │        └─ 返回文档列表
    │
    ├─ 3.2 文档分块
    │    └─ knowledge/chunker.py →
DocumentChunker.split_documents(documents)
    │        ├─ 读取配置的分块策略（recursive 或markdown）
    │        ├─ 如果是recursive:
    │        │    └─ 创建RecursiveCharacterTextSplitter
    │        │        ├─ 按段落切分（\n\n）
    │        │        ├─ 按行切分（\n）
    │        │        ├─ 按句子切分（。！？）
    │        │        └─ 得到500字的块，重叠50字
    │        ├─ 如果是markdown:
    │        │    └─ 创建MarkdownHeaderTextSplitter
    │        │        ├─ 按标题切分（#, ##, ###）
    │        │        ├─ 每个标题+内容是一个块
    │        │        ├─ 超长块用递归方法二次切分
    │        │        └─ 保持文档结构
    │        └─ 返回分块列表
    │
    └─ 3.3 添加到分块列表
         └─ 收集所有分块
    ↓
【第4步】存入向量库
    └─ vector_db.add_documents(all_chunks)
         ├─ 对每个分块计算嵌入向量（使用嵌入模型）
         ├─ 将向量+元数据+内容存入Chroma数据库
         └─ 持久化到磁盘
    ↓
【第5步】重新初始化BM25检索器
    └─ knowledge/knowledge_base.py → init_bm25_retriever(all_docs)
         ├─ 从向量库获取所有文档
         ├─ 创建BM25索引
         └─ 返回BM25检索器
    ↓
【第6步】重新构建检索器
    └─ retrieval/retriever.py → build_final_retriever(...)
         ├─ 现在bm25_retriever不为None了
         ├─ 创建向量检索器
         ├─ 创建混合检索器（向量+BM25）
         ├─ 应用重排序器
         └─ 返回最终检索器
```

```
    ↓
【第7步】清除缓存并刷新页面
    └─ st.cache_resource.clear() + st.rerun()
    ↓
显示更新成功消息
```

2.3 后续运行（向量库已存在）

```
用户运行: streamlit run main.py
    ↓
【第1步】加载配置
    └─ 读取 config.py
    ↓
【第2步】初始化系统组件
    ↓
    ├─ 2.1 初始化嵌入模型
    │   └─ init_embeddings()
    │       └─ 返回 embeddings 对象
    │
    ├─ 2.2 初始化大模型
    │   └─ init_llm()
    │       └─ 返回 llm 对象
    │
    ├─ 2.3 加载向量库（不是首次）
    │   └─ knowledge/knowledge_base.py → init_vector_db(embeddings)
    │       ├─ 检查 Chroma 数据库路径存在
    │       ├─ 从磁盘加载已有的向量库
    │       ├─ 获取所有已存储的文档和元数据
    │       └─ 返回 vector_db 和 all_docs
    │
    ├─ 2.4 初始化BM25检索器
    │   └─ init_bm25_retriever(all_docs)
    │       ├─ all_docs 不为空
    │       ├─ 创建BM25索引
    │       └─ 返回 BM25检索器
    │
    ├─ 2.5 初始化重排序器
    │   └─ init_reranker()
    │       └─ 返回 reranker 对象
    │
    └─ 2.6 构建最终检索器
        └─ build_final_retriever(vector_db, bm25_retriever, reranker)
            ├─ 创建向量检索器
            ├─ 创建混合检索器（向量+BM25）
```

```
                ├─ 应用重排序器
                └─ 返回最终检索器
    ↓
【第3步】创建问答链
    └─ QAChain(llm, final_retriever)
    ↓
【第4步】显示Web界面
    └─ 显示界面，等待用户操作
```

---

3. 在线阶段：问答流程

3.1 用户提问（单轮对话）

```
用户在输入框输入："什么是RAG？"并按回车
    ↓
【第1步】main.py 捕获用户输入
    └─ user_question = st.chat_input("请输入你的问题...")
    ↓
【第2步】显示用户消息
    └─ st.chat_message("user")
    └─ st.markdown(user_question)
    ↓
【第3步】调用问答链
    └─ result = qa_chain.answer(
        question=user_question,
        chat_history=[]  # 单轮对话，历史为空
    )
    ↓
【第4步】进入QAChain.answer()方法（generation/qa_chain.py）
    ↓
    └─ 4.1 检查是否需要问题改写
        └─ if len(chat_history) > 0:  # 历史为空，不需要改写
            rewritten_question = None
            search_query = user_question
    ↓
    └─ 4.2 检索文档
        └─ docs = final_retriever.invoke(search_query)
            ↓
    进入检索器链路（retrieval/retriever.py）：
            ├─ 步骤1：向量检索
            │   └─ vector_retriever.invoke("什么是RAG？")
            │       ├─ 将问题转换为向量（使用嵌入模型）
```

```
        │        ├─ 在向量库中搜索最相似的10个块（TOP_K_WIDE=10）
        │        └─ 返回10个文档片段
        │
        ├─ 步骤2：混合检索（如果启用）
        │   └─ if USE_MIXED_RETRIEVAL:  # 假设启用
        │        ├─ BM25检索：用BM25算法搜索关键词
        │        ├─ 得到10个文档片段
        │        └─ 合并向量检索和BM25结果
        │            └─ 权重：向量0.7，BM25 0.3
        │            └─ 综合排序，返回10个文档片段
        │
        └─ 步骤3：重排序（如果启用）
            └─ if USE_RERANKER:  # 假设启用
                 ├─ 将10个文档片段输入重排序模型
                 ├─ 交叉编码器计算每个文档与问题的相关性分数
                 ├─ 示例：
                 │   文档1：9.5分（高度相关）
                 │   文档2：8.8分
                 │   文档3：8.2分
                 │   ...
                 │   文档10：6.1分
                 └─ 选择Top-3（TOP_K_FINAL=3）
                     └─ 返回最相关的3个文档片段
        ↓
返回docs（3个文档片段）
↓
└─ 4.3 格式化源文档（generation/qa_chain.py）
    └─ context = format_source_docs(docs)
        └─ 遍历每个文档，格式化为：
  【来源：file1.pdf 第3页】
            RAG（Retrieval-Augmented Generation）是...

  【来源：file2.md 第1页】
  检索增强生成技术结合了...
↓
└─ 4.4 构建提示词（generation/prompts.py）
    └─ prompt = SINGLE_TURN_PROMPT.format(
        context=context,
        question=user_question
    )
        ↓
得到完整提示词：
你是一个专业的文档问答助手，必须严格遵守以下规则：

        1. 仅基于【检索到的文档内容】回答用户的问题
        2. 如果没有相关信息，直接说"找不到"
        ...
```

```
    【检索到的文档内容】:
    【来源: file1.pdf 第3页】
           RAG(Retrieval-Augmented Generation)是...

    【来源: file2.md 第1页】
    检索增强生成技术结合了...

    【用户的问题】:
    什么是RAG?

    【你的回答】:
    ↓
    └─ 4.5 调用LLM生成答案
        └─ answer = llm.invoke(prompt).strip()
            ↓
    进入Ollama模型(本地运行)
           ├─ 接收提示词
           ├─ 模型推理
           └─ 返回答案:
               RAG(Retrieval-Augmented Generation)是一种结合了
      信息检索和文本生成的AI技术, 通过从外部知识库中
      检索相关信息来增强大语言模型的生成能力...

    ↓
【第5步】返回结果
    └─ return {
        "answer": answer,
        "source_docs": docs,
        "rewritten_question": None
    }
    ↓
【第6步】main.py 处理返回结果
    ↓
    ├─ 6.1 显示助手消息
    │   └─ st.chat_message("assistant")
    │   └─ st.markdown(result["answer"])
    │       └─ 显示:
    │           RAG(Retrieval-Augmented Generation)是一种结合了
    │           信息检索和文本生成的AI技术...
    │
    ├─ 6.2 显示源文档
    │   └─ with st.expander("📄 查看检索到的源文档片段")
    │       └─ for doc in docs:
    │           ├─ 显示元数据: file1.pdf 第3页
    │           └─ 显示内容: RAG(Retrieval-Augmented Generation)...
    │
```

```
    └─ 6.3 保存到对话历史
        └─ st.session_state.messages.append({
            "role": "assistant",
            "content": result["answer"],
            "source_docs": docs
        })
    ↓
等待下一次提问
```

3.2 用户提问（多轮对话）

```
对话历史:
用户: 什么是RAG?
助手: RAG是检索增强生成...

用户提问: "它的优点是什么？"
    ↓
【第1步】main.py 捕获用户输入
    └─ user_question = "它的优点是什么？"
    ↓
【第2步】调用问答链
    └─ result = qa_chain.answer(
        question="它的优点是什么？",
        chat_history=st.session_state.messages[:-1]  # 传入之前的对话历史
    )
    ↓
【第3步】进入QAChain.answer()方法（generation/qa_chain.py）
    ↓
    └─ 3.1 格式化对话历史
        └─ formatted_history = format_chat_history(chat_history)
            └─ 格式化为:
                user: 什么是RAG?
                assistant: RAG是检索增强生成...
    ↓
    └─ 3.2 问题改写（关键步骤！）
        └─ if len(chat_history) > 0:  # 有历史，需要改写
            ↓
            prompt = QUERY_REWRITE_PROMPT.format(
                chat_history=formatted_history,
                question="它的优点是什么？"
            )
                ↓
      得到问题改写提示词:
      请结合【对话历史】，把用户的【当前问题】改写成一个完整、无指代的问题。
```

```
  【对话历史】:
            user: 什么是RAG?
            assistant: RAG是检索增强生成...

  【用户的当前问题】:
  它的优点是什么?

  【改写后的问题】:
        ↓
        rewritten_question = llm.invoke(prompt).strip()
            ↓
            LLM理解上下文, 返回:
            "RAG的优点是什么? "
            ↑
            "它"被替换为"RAG"
        ↓
        search_query = "RAG的优点是什么? "
↓
└─ 3.3 检索文档
    └─ docs = final_retriever.invoke(search_query)
        ↓
用改写后的问题"RAG的优点是什么? "检索
        ├─ 向量检索: 搜索"RAG的优点"
        ├─ BM25检索: 搜索关键词"RAG""优点"
        ├─ 混合检索: 合并结果
        └─ 重排序: 选出最相关的3个文档
        ↓
返回3个关于RAG优点的文档片段
↓
└─ 3.4 格式化源文档
    └─ context = format_source_docs(docs)
↓
└─ 3.5 构建提示词(使用多轮对话模板)
    └─ prompt = MULTI_TURN_PROMPT.format(
        chat_history=formatted_history,
        context=context,
        question="它的优点是什么? "  # 原问题, 不是改写后的
    )
        ↓
得到完整提示词:
你是一个专业的文档问答助手...

【对话历史】:
        user: 什么是RAG?
        assistant: RAG是检索增强生成...
```

```
        【检索到的文档内容】:
        【来源: file1.pdf 第5页】
                RAG的优点包括: 1. 提高回答准确性...
        【来源: file2.md 第2页】
                RAG的主要优势: 减少幻觉、知识更新...

        【用户的当前问题】:
        它的优点是什么?

        【你的回答】:
        ↓
        └─ 3.6 调用LLM生成答案
            └─ answer = llm.invoke(prompt).strip()
                ↓
                LLM结合历史和文档, 返回:
                RAG的优点主要包括:
                1. 提高回答准确性: 通过检索权威资料...
                2. 减少幻觉: 基于真实文档生成...
                3. 知识更新: 可以动态更新知识库...
        ↓
        └─ 3.7 返回结果
            └─ return {
                "answer": answer,
                "source_docs": docs,
                "rewritten_question": "RAG的优点是什么? "
            }
        ↓
【第4步】main.py 处理返回结果
        ↓
        ├─ 显示答案
        ├─ 显示源文档
        └─ 保存到对话历史(包括改写后的问题)
        ↓
等待下一次提问
```

---

4. 各文件内部执行顺序

4.1 main.py 内部执行顺序

```
main.py 启动
    ↓
【第1步】页面配置
```

```
    └─ st.set_page_config()
        ├─ page_title: "完整RAG问答系统"
        ├─ page_icon: "📚"
        └─ layout: "wide"
    ↓
【第2步】显示标题
    └─ st.title("📚 完整RAG文档问答系统")
    └─ st.caption("基于LangChain + Chroma + Ollama...")
    ↓
【第3步】导入依赖
    ├─ import config
    ├─ import streamlit as st
    ├─ from utils import check_ollama_running
    ├─ from knowledge import init_embeddings, init_vector_db,
build_or_update_knowledge_base, init_bm25_retriever
    ├─ from retrieval import build_final_retriever
    ├─ from retrieval.reranker import init_reranker
    ├─ from generation import QAChain
    └─ from generation.prompts import format_chat_history
    ↓
【第4步】初始化会话状态
    └─ if "messages" not in st.session_state:
        └─ st.session_state.messages = []
    ↓
【第5步】初始化系统组件（带缓存）
    └─ with st.spinner("正在初始化系统组件..."):
        ├─ @st.cache_resource
        ├─ def init_system_components():
        │   ├─ embeddings = init_embeddings()
        │   ├─ llm = Ollama(model=config.LLM_MODEL)
        │   ├─ reranker = init_reranker() if config.USE_RERANKER else
None
        │   ├─ vector_db, all_docs = init_vector_db(embeddings)
        │   ├─ bm25_retriever = init_bm25_retriever(all_docs)
        │   ├─ final_retriever = build_final_retriever(vector_db,
bm25_retriever, reranker)
        │   └─ qa_chain = QAChain(llm, final_retriever)
        │   └─ return components
        │
        └─ components = init_system_components()
    ↓
【第6步】渲染侧边栏
    └─ with st.sidebar:
        ├─ st.header("⚙️ 系统配置与状态")
        │
        ├─ # 系统状态检查
        │   with st.expander("系统状态检查", expanded=True):
```

```
        │   ├─ status, msg = check_ollama_running()
        │   ├─ if status: st.success(msg)
        │   └─ else: st.error(msg)
        │
        ├─ st.divider()
        │
        ├─ # 知识库管理
        │   st.header("📂 知识库管理")
        │   ├─ kb_path = config.KNOWLEDGE_BASE_DIR
        │   ├─ st.info(f"知识库文件夹: {kb_path}")
        │   │
        │   ├─ # 显示知识库统计
        │   │   vector_db = components["vector_db"]
        │   │   all_docs = vector_db.get()
        │   │   ├─ doc_count = len(set([m['file_md5'] for m in
all_docs['metadatas']]))
        │   │   └─ chunk_count = len(all_docs['ids'])
        │   │
        │   │   st.info(f"当前知识库文档数量: {doc_count} 个")
        │   │   st.info(f"当前知识库分块数量: {chunk_count} 个")
        │   │
        │   └─ # 更新知识库按钮
        │       if st.button("🔄 更新知识库", type="primary",
use_container_width=True):
        │           with st.spinner("正在更新知识库..."):
        │               embeddings = components["embeddings"]
        │               vector_db = components["vector_db"]
        │               all_docs = vector_db.get()
        │               chunks, msg =
build_or_update_knowledge_base(embeddings, vector_db, all_docs)
        │               if chunks:
        │                   st.success(msg)
        │                   st.cache_resource.clear()
        │                   st.rerun()
        │               else:
        │                   st.warning(msg)
        │
        └─ st.divider()
    ↓
【第7步】渲染主界面
    └─ # 显示历史对话
        for msg in st.session_state.messages:
            with st.chat_message(msg["role"]):
                st.markdown(msg["content"])
                if "source_docs" in msg:
                    with st.expander("📄 查看检索到的源文档片段"):
                        for doc in msg["source_docs"]:
```

```
                                file_name = doc.metadata.get('file_name',
'未知')
                                page = doc.metadata.get('page', '未知')
                                st.markdown(f"**【源文档: {file_name} 第
{page}页】**")
                                st.code(doc.page_content, language="text")
    ↓
【第8步】获取用户输入
    └─ user_question = st.chat_input("请输入你的问题...")
    ↓
【第9步】处理用户提问（如果有）
    └─ if user_question:
        ├─ # 显示用户消息
        │   with st.chat_message("user"):
        │       st.markdown(user_question)
        │
        ├─ # 添加到历史
        │   st.session_state.messages.append({
        │       "role": "user",
        │       "content": user_question
        │   })
        │
        ├─ # 获取组件
        │   qa_chain = components["qa_chain"]
        │
        ├─ # 调用问答链
        │   with st.chat_message("assistant"):
        │       with st.spinner("正在检索文档并生成回答..."):
        │           result = qa_chain.answer(
        │               question=user_question,
        │               chat_history=st.session_state.messages[:-1]
        │           )
        │
        │       # 显示回答
        │       st.markdown(result["answer"])
        │
        │       # 显示源文档
        │       with st.expander("📄 查看检索到的源文档片段"):
        │           for i, doc in enumerate(result["source_docs"]):
        │               file_name = doc.metadata.get('file_name', '未
知')
        │               page = doc.metadata.get('page', '未知')
        │               st.markdown(f"**【源文档 {i+1}: {file_name} 第
{page}页】**")
        │               st.code(doc.page_content, language="text")
        │
        │       # 显示改写后的问题（多轮对话时）
```

```
        │       if result["rewritten_question"]:
        │           st.caption(f"📝 问题改写:
{result['rewritten_question']}")
        │
        ├─ # 保存到历史
        │   st.session_state.messages.append({
        │       "role": "assistant",
        │       "content": result["answer"],
        │       "source_docs": result["source_docs"],
        │       "rewritten_question": result["rewritten_question"]
        │   })
        │
        └─ st.rerun()  # 刷新页面，显示新消息
    ↓
【循环】等待下一次用户输入
```

4.2 config.py 内部执行顺序

```
config.py 被导入
    ↓
【第1步】导入标准库
    └─ import os
    └─ from dotenv import load_dotenv
    ↓
【第2步】加载环境变量
    └─ load_dotenv()
        └─ 从 .env 文件加载环境变量（如果存在）
    ↓
【第3步】定义模型配置
    ├─ EMBEDDING_MODEL = "BAAI/bge-small-zh-v1.5"
    ├─ RERANKER_MODEL = "BAAI/bge-reranker-base"
    ├─ LLM_MODEL = "qwen2.5:3b"
    └─ DEVICE = "cpu"
    ↓
【第4步】定义知识库配置
    ├─ KNOWLEDGE_BASE_DIR = r"D:\knowledge_base"
    ├─ CHROMA_DB_PATH = r"D:\chroma_db"
    ├─ CHUNK_SIZE = 500
    ├─ CHUNK_OVERLAP = 50
    ├─ CHUNK_STRATEGY = "recursive"
    └─ SUPPORTED_FORMATS = [".pdf", ".docx", ".md", ".txt"]
    ↓
【第5步】定义检索配置
    ├─ USE_MIXED_RETRIEVAL = True
```

```
    ├─ USE_RERANKER = True
    ├─ TOP_K_WIDE = 10
    ├─ TOP_K_FINAL = 3
    ├─ VECTOR_WEIGHT = 0.7
    └─ BM25_WEIGHT = 0.3
    ↓
【第6步】定义对话配置
    ├─ USE_MULTI_TURN = True
    └─ LLM_TEMPERATURE = 0.1
    ↓
【第7步】定义界面配置
    ├─ PAGE_TITLE = "完整RAG问答系统"
    ├─ PAGE_ICON = "📚"
    └─ LAYOUT = "wide"
    ↓
【完成】所有配置变量定义完成，可供其他模块导入使用
```

4.3 utils/file_utils.py 内部执行顺序

```
utils/file_utils.py 被导入
    ↓
【第1步】导入依赖
    └─ import os
    └─ import hashlib
    └─ from datetime import datetime
    ↓
【第2步】定义函数（按调用顺序）
    ↓
函数1: get_file_md5(file_path)
    ├─ 接收文件路径
    ├─ 创建MD5哈希对象: hash_md5 = hashlib.md5()
    ├─ 以二进制模式打开文件: with open(file_path, "rb") as f:
    ├─ 逐块读取（每次4096字节）:
    │   for chunk in iter(lambda: f.read(4096), b""):
    │       hash_md5.update(chunk)
    └─ 返回32位十六进制字符串
    ↓
函数2: get_file_extension(file_path)
    ├─ 接收文件路径
    ├─ 提取扩展名: ext = os.path.splitext(file_path)[1]
    └─ 返回小写的扩展名
    ↓
函数3: get_file_name(file_path)
    ├─ 接收文件路径
```

```
    ├─ 提取文件名（含扩展名）: name = os.path.basename(file_path)
    └─ 返回文件名
    ↓
【完成】所有函数定义完成
```

4.4 utils/system_utils.py 内部执行顺序

```
utils/system_utils.py 被导入
    ↓
【第1步】导入依赖
    └─ from langchain_community.llms import Ollama
    └─ import config
    ↓
【第2步】定义函数
    ↓
函数: check_ollama_running()
    ├─ try:
    │   ├─ 创建Ollama实例: llm = Ollama(model=config.LLM_MODEL)
    │   ├─ 发送测试请求: llm.invoke("你好")
    │   └─ 返回 (True, "Ollama运行正常, 模型可用")
    ├─ except Exception as e:
    │   └─ 返回 (False, f"Ollama检查失败: {str(e)}")
    └─ 完成
```

4.5 knowledge/document_loader.py 内部执行顺序

```
knowledge/document_loader.py 被导入
    ↓
【第1步】导入依赖
    ├─ import os
    ├─ from langchain_community.document_loaders import (
    │   PyPDFLoader,
    │   Docx2txtLoader,
    │   UnstructuredMarkdownLoader,
    │   TextLoader
    │   )
    ├─ from datetime import datetime
    ├─ from utils import (
    │   get_file_md5,
    │   get_file_extension,
    │   get_file_name
```

```
    │   )
    └─ from config import SUPPORTED_FORMATS
    ↓
【第2步】定义类和函数
    ↓
类: DocumentLoader
    ↓
  方法1: __init__(self, supported_formats=None)
         ├─ 接收支持的格式列表
         ├─ 如果为空，使用默认: SUPPORTED_FORMATS
         └─ 保存到 self.supported_formats
    ↓
  方法2: _is_supported(self, file_path)
         ├─ 获取文件扩展名: get_file_extension(file_path)
         ├─ 判断是否在支持列表中
         └─ 返回 True/False
    ↓
  方法3: _get_loader(self, file_path)
         ├─ 获取文件扩展名
         ├─ 根据扩展名选择加载器:
         │   ├─ .pdf → PyPDFLoader(file_path)
         │   ├─ .docx → Docx2txtLoader(file_path)
         │   ├─ .md → UnstructuredMarkdownLoader(file_path)
         │   └─ .txt → TextLoader(file_path)
         └─ 返回加载器实例
    ↓
  方法4: _add_metadata(self, documents, file_path)
         ├─ 计算文件MD5: file_md5 = get_file_md5(file_path)
         ├─ 获取文件名: file_name = get_file_name(file_path)
         ├─ 获取当前时间: update_time = datetime.now().strftime(...)
         ├─ 遍历每个文档:
         │   ├─ 添加元数据: doc.metadata["file_name"] = file_name
         │   ├─ doc.metadata["file_md5"] = file_md5
         │   └─ doc.metadata["update_time"] = update_time
         └─ 完成
    ↓
  方法5: load_document(self, file_path)  ← 主要方法
         ├─ 检查文件是否支持: _is_supported(file_path)
         ├─ 如果不支持，抛出异常
         ├─ 获取加载器: _get_loader(file_path)
         ├─ 加载文档: documents = loader.load()
         ├─ 添加元数据: _add_metadata(documents, file_path)
         └─ 返回文档列表
    ↓
  方法6: load_documents(self, file_paths)
         ├─ 初始化结果列表: all_documents = []
         ├─ 初始化失败列表: failed_files = []
```

```
        ├─ 遍历每个文件路径:
        │   ├─ try:
        │   │   ├─ 调用 load_document(file_path)
        │   │   └─ 扩展到 all_documents
        │   └─ except Exception as e:
        │       └─ 记录失败: failed_files.append(...)
        └─ 返回 (all_documents, failed_files)
    ↓
【完成】类定义完成
```

4.6 knowledge/chunker.py 内部执行顺序

```
knowledge/chunker.py 被导入
    ↓
【第1步】导入依赖
    ├─ from langchain.text_splitter import (
    │   RecursiveCharacterTextSplitter,
    │   MarkdownHeaderTextSplitter
    │   )
    └─ from config import CHUNK_SIZE, CHUNK_OVERLAP, CHUNK_STRATEGY
    ↓
【第2步】定义类
    ↓
类: DocumentChunker
    ↓
 方法1: __init__(self, chunk_size, chunk_overlap, strategy)
        ├─ 接收分块参数
        └─ 保存到实例变量
    ↓
 方法2: _split_by_recursive(self, documents)
        ├─ 创建递归分块器:
        │   splitter = RecursiveCharacterTextSplitter(
        │       chunk_size=self.chunk_size,
        │       chunk_overlap=self.chunk_overlap,
        │       separators=["\n\n", "\n", "。", "! ", "? ", " ", ""]
        │   )
        ├─ 对每个文档进行分块:
        │   for doc in documents:
        │       chunks = splitter.split_documents([doc])
        │       all_chunks.extend(chunks)
        └─ 返回所有分块
    ↓
 方法3: _split_by_markdown(self, documents)
        ├─ 创建Markdown标题分块器:
```

```
          │   header_splitter = MarkdownHeaderTextSplitter(
          │       headers_to_split_on=[
          │           ("#", "一级标题"),
          │           ("##", "二级标题"),
          │           ("###", "三级标题")
          │       ]
          │   )
          ├─ 第一轮: 按标题分块
          │   ├─ for doc in documents:
          │   │   chunks = header_splitter.split_text(doc.page_content)
          │   │   for chunk in chunks:
          │   │       chunk.metadata.update(doc.metadata)
          │   │   split_docs.extend(chunks)
          ├─ 第二轮: 递归分块（处理超长块）
          │   ├─ 创建递归分块器
          │   ├─ 对第一轮结果进行二次分块
          │   └─ 返回最终分块
          └─ 完成
    ↓
  方法4: split_documents(self, documents)  ← 主要方法
          ├─ 判断分块策略:
          │   ├─ if strategy == "markdown":
          │   │   └─ return self._split_by_markdown(documents)
          │   └─ else:
          │       └─ return self._split_by_recursive(documents)
    ↓
【完成】类定义完成
```

4.7 knowledge/knowledge_base.py 内部执行顺序

文件结构概览

```
knowledge/knowledge_base.py (262行)
├── 导入依赖
│   ├── langchain_chroma.Chroma (注意: 新版导入路径)
│   ├── langchain_community.retrievers.BM25Retriever
│   ├── langchain_core.documents.Document
│   ├── config (配置参数)
│   ├── utils.file_utils (工具函数)
│   ├── .document_loader (相对导入)
│   └── .chunker (相对导入)
├── 类定义: KnowledgeBase
│   ├── __init__()                  # 类初始化
│   ├── load_vector_db()            # 加载/创建向量库
```

```
│    ├── _init_bm25_retriever()       # 私有方法：初始化BM25
│    ├── update_knowledge_base()     # 更新知识库
│    ├── _scan_knowledge_folder()     # 私有方法：扫描文件夹
│    ├── _filter_files_to_update()    # 私有方法：筛选文件
│    ├── _process_files()             # 私有方法：批量处理
│    ├── get_vector_db()              # 便捷方法：获取向量库
│    ├── get_bm25_retriever()          # 便捷方法：获取BM25
│    └── get_stats()                  # 便捷方法：获取统计
└── 注意：没有顶层便捷函数（所有功能都封装在类中）
```

完整执行顺序

```
knowledge/knowledge_base.py 被导入时
    ↓
【第1步】导入依赖模块
    ├─ import os
    ├─ from langchain_chroma import Chroma
    │   # ⚠️注意：使用langchain_chroma（新版），而不是langchain_community
    │   # 原因：LangChain 0.1.0+ 版本将Chroma 独立为单独的包
    │   # 影响：如果安装旧版LangChain，需要改用langchain_community
    │
    ├─ from langchain_community.retrievers import BM25Retriever
    │   # BM25：基于关键词的检索算法
    │   # 作用：与向量检索互补，提高召回率
    │
    ├─ from langchain_core.documents import Document
    │   # Document：LangChain的文档对象
    │   # 结构：Document(page_content="...", metadata={...})
    │
    ├─ from config import (
    │   KNOWLEDGE_BASE_DIR,     # 知识库文件夹路径
    │   CHROMA_DB_PATH,         # 向量库路径
    │   SUPPORTED_FORMATS,      # 支持的文件格式
    │   TOP_K_WIDE,             # 宽召回数量
    │   USE_MIXED_RETRIEVAL     # 是否使用混合检索
    │   )
    │
    ├─ from utils.file_utils import (
    │   get_file_md5,           # 计算文件MD5（用于增量更新）
    │   get_file_name,          # 获取文件名
    │   is_supported_format,    # 检查文件格式
    │   ensure_directory_exists  # 确保目录存在
    │   )
    │
```

```
    ├─ from .document_loader import DocumentLoader
    │   # 相对导入：.表示当前目录（knowledge/）
    │   # 导入文档加载器类
    │
    └─ from .chunker import DocumentChunker
        # 相对导入：导入文档分块器类
    ↓
【第2步】定义类：KnowledgeBase（知识库管理类）
    ↓
    ├── 【2.1】类初始化：__init__()
    │   │
    │   ├─ 接收参数：
    │   │   ├─ embedding_model：嵌入模型对象（必选）
    │   │   └─ vector_db_path：向量库路径（可选，默认CHROMA_DB_PATH）
    │   │
    │   ├─ 保存实例变量：
    │   │   ├─ self.embedding_model = embedding_model
    │   │   │   # 作用：后续用于计算文档嵌入向量
    │   │   │
    │   │   ├─ self.vector_db_path = vector_db_path
    │   │   │   # 作用：向量库存储位置，如 "D:/chroma_db"
    │   │   │
    │   │   ├─ self.vector_db = None
    │   │   │   # 作用：向量数据库对象（稍后在load_vector_db() 中初始化）
    │   │   │   # ⚠️歧义点：这里只是占位符，实际对象在后面才创建
    │   │   │
    │   │   ├─ self.all_docs = None
    │   │   │   # 作用：存储所有文档的元数据
    │   │   │   # 结构：{"documents": [...], "metadatas": [...], "ids":
[...]}
    │   │   │   # ⚠️歧义点：documents 和metadatas 是分开存储的
    │   │   │
    │   │   └─ self.bm25_retriever = None
    │   │       # 作用：BM25检索器对象（稍后初始化）
    │   │       # 如果没有文档，此值为None
    │   │
    │   └─ 初始化子组件：
    │       ├─ self.document_loader = DocumentLoader()
    │       │   # 作用：负责加载各种格式的文档
    │       │   # 创建实例时不需要参数
    │       │
    │       └─ self.document_chunker = DocumentChunker()
    │           # 作用：负责文档分块
    │           # 创建实例时会读取配置参数（CHUNK_SIZE等）
    │
    ↓
    ├── 【2.2】公共方法：load_vector_db() ← 加载或创建向量数据库
```

```
    │   │
    │   ├─ 调用时机:
    │   │   ├─ 首次启动时: 创建新的空向量库
    │   │   └─ 后续启动时: 加载已有的向量库
    │   │
    │   ├─ 执行步骤:
    │   │   │
    │   │   ├─ 【步骤1】检查向量库目录
    │   │   │   condition = os.path.exists(self.vector_db_path) and
len(os.listdir(self.vector_db_path)) > 0
    │   │   │   # ⚠️ 歧义点: 不仅要检查目录存在, 还要检查目录不为空
    │   │   │   # 原因: Chroma可能创建了空目录
    │   │   │
    │   │   ├─ 【情况A: 向量库已存在且不为空】
    │   │   │   │
    │   │   │   ├─ 加载已有向量库:
    │   │   │   │   self.vector_db = Chroma(
    │   │   │   │       persist_directory=self.vector_db_path,
    │   │   │   │       embedding_function=self.embedding_model
    │   │   │   │   )
    │   │   │   │   # persist_directory: 向量库持久化路径
    │   │   │   │   # embedding_function: 用于计算嵌入的模型
    │   │   │   │   # ⚠️ 歧义点: 必须传入相同的embedding_model, 否则无法检索
    │   │   │   │
    │   │   │   ├─ 获取所有文档数据:
    │   │   │   │   self.all_docs = self.vector_db.get()
    │   │   │   │   # Chroma.get() 返回结构:
    │   │   │   │   # {
    │   │   │   │   #     "ids": ["chunk_0", "chunk_1", ...],
    │   │   │   │   #     "documents": ["文档1内容", "文档2内容", ...],
    │   │   │   │   #     "metadatas": [{"file_name":
"a.pdf", ...}, ...],
    │   │   │   │   #     "embeddings": [[...], [...], ...]
    │   │   │   │   # }
    │   │   │   │   # ⚠️ 歧义点: 新版比旧版多了 "ids" 字段
    │   │   │   │
    │   │   │   └─ 作用: 用于增量更新(对比MD5)
    │   │   │
    │   │   └─ 【情况B: 向量库不存在或为空】
    │   │       │
    │   │       ├─ 创建新的空向量库:
    │   │       │   self.vector_db = Chroma(
    │   │       │       persist_directory=self.vector_db_path,
    │   │       │       embedding_function=self.embedding_model
    │   │       │       )
    │   │       │   # Chroma会自动创建目录结构
    │   │       │
```

```
│   │   │   ├─ 初始化空文档数据:
│   │   │   │   self.all_docs = {
│   │   │   │       "documents": [],
│   │   │   │       "metadatas": [],
│   │   │   │       "ids": []
│   │   │   │   }
│   │   │   │   # ⚠️ 歧义点: 三个字段都必须初始化为空列表
│   │   │   │
│   │   │   └─ 作用: 标记为"空库", 后续会调用 update_knowledge_base()
│   │   │
│   │   ├─ 【步骤2】初始化BM25检索器
│   │   │   self._init_bm25_retriever()
│   │   │   # ⚠️ 注意: 这里调用私有方法(以下划线开头)
│   │   │   # 私有方法的意思: 建议只在类内部使用, 外部不要直接调用
│   │   │
│   │   └─ 【返回】(self.vector_db, self.all_docs)
│   │       # 返回元组, 供外部使用
│   │       # 作用: 外部可以获取向量库对象和文档数据
│   │
│   ↓
│   ├── 【2.3】私有方法: _init_bm25_retriever() ← 初始化BM25检索器
│   │   │
│   │   ├─ 方法命名规则:
│   │   │   └─ 以下划线 _ 开头, 表示私有方法
│   │   │       # Python中这不是强制的, 只是一种约定
│   │   │       # 意图: 告诉其他开发者, 这是内部方法, 不要直接调用
│   │   │
│   │   ├─ 执行步骤:
│   │   │   │
│   │   │   ├─ 【步骤1】检查是否有文档
│   │   │   │   if not self.all_docs["documents"]:
│   │   │   │       self.bm25_retriever = None
│   │   │   │       return
│   │   │   │   # ⚠️ 歧义点: 检查的是 documents 字段, 不是 metadatas
│   │   │   │   # 原因: documents 是主要字段, 如果为空说明没文档
│   │   │   │
│   │   │   ├─ 【步骤2】从向量库数据恢复Document对象
│   │   │   │   # ⚠️ 歧义点: 为什么要恢复Document对象?
│   │   │   │   # 原因: BM25Retriever需要Document对象, 而Chroma返回的是字典
│   │   │   │   # 注意: 这里不包含嵌入向量, BM25只需要文本
│   │   │   │
│   │   │   │   bm25_docs = [
│   │   │   │       Document(page_content=doc, metadata=meta)
│   │   │   │       for doc, meta in zip(
│   │   │   │           self.all_docs["documents"],
│   │   │   │           self.all_docs["metadatas"]
│   │   │   │       )
```

```
    │   │   │   ]
    │   │   │   # zip(): 将两个列表一一对应组合
    │   │   │   # 结果: [(doc1, meta1), (doc2, meta2), ...]
    │   │   │   # Document(): 创建Document对象
    │   │   │
    │   │   ├─ 【步骤3】创建BM25检索器
    │   │   │   self.bm25_retriever =
BM25Retriever.from_documents(bm25_docs)
    │   │   │   # from_documents(): 从文档列表创建BM25索引
    │   │   │   # 内部处理:
    │   │   │   # 1. 分词: 对每个文档进行分词
    │   │   │   # 2. 建立索引: 记录每个词在哪些文档中出现
    │   │   │   # 3. 计算权重: TF-IDF算法
    │   │   │   # ⚠️歧义点: BM25在创建时就会建立索引, 比较耗时
    │   │   │
    │   │   └─ 【步骤4】设置检索数量
    │   │       self.bm25_retriever.k = TOP_K_WIDE
    │   │       # TOP_K_WIDE: 通常为10
    │   │       # 作用: 每次检索返回10个最相关的文档
    │   │       # ⚠️歧义点: 这里设置的是BM25的k值, 不是最终的k值
    │   │       # 最终的k值由重排序器决定(TOP_K_FINAL=3)
    │
    ↓
    ├── 【2.4】公共方法: update_knowledge_base() ← 增量更新知识库
    │   │
    │   ├─ 调用时机:
    │   │   └─ 用户点击"更新知识库"按钮时
    │   │
    │   ├─ 核心思想: 增量更新
    │   │   ├─ 只处理新增或修改的文件
    │   │   ├─ 通过MD5值判断文件是否变化
    │   │   └─ 避免重复处理, 提高效率
    │   │
    │   ├─ 执行步骤:
    │   │   │
    │   │   ├─ 【步骤1】扫描知识库文件夹
    │   │   │   target_files = self._scan_knowledge_folder()
    │   │   │   ↓
    │   │   │   # 调用私有方法 _scan_knowledge_folder()
    │   │   │   # 该方法会遍历文件夹, 收集所有支持的文件
    │   │   │   # 返回: ["D:/kb/file1.pdf", "D:/kb/file2.docx", ...]
    │   │   │
    │   │   ├─ 【步骤2】检查是否有文件
    │   │   │   if not target_files:
    │   │   │       return [], "知识库文件夹中没有找到支持的文档"
    │   │   │   # 如果文件夹为空, 直接返回
    │   │   │
```

```
    │   │   ├─ 【步骤3】筛选需要更新的文件
    │   │   │   pending_files =
self._filter_files_to_update(target_files)
    │   │   │   ↓
    │   │   │   # 调用私有方法 _filter_files_to_update()
    │   │   │   # 该方法会对比文件MD5，找出新增或修改的文件
    │   │   │   # 返回: ["D:/kb/new_file.pdf", ...]
    │   │   │
    │   │   ├─ 【步骤4】检查是否有需要更新的文件
    │   │   │   if not pending_files:
    │   │   │       return [], "知识库已是最新，没有新增/修改的文档"
    │   │   │   # 如果没有新文件，直接返回
    │   │   │
    │   │   ├─ 【步骤5】批量处理文件
    │   │   │   all_chunks, success_count, fail_count, fail_files =
self._process_files(pending_files)
    │   │   │   ↓
    │   │   │   # 调用私有方法 _process_files()
    │   │   │   # 该方法会逐个加载、分块文件
    │   │   │   # 返回: (所有分块, 成功数, 失败数, 失败文件列表)
    │   │   │
    │   │   ├─ 【步骤6】存入向量库
    │   │   │   if all_chunks:
    │   │   │       self.vector_db.add_documents(all_chunks)
    │   │   │       # Chroma.add_documents() 会自动处理:
    │   │   │       # 1. 计算嵌入向量（使用self.embedding_model）
    │   │   │       # 2. 存储向量、内容、元数据
    │   │   │       # 3. 持久化到磁盘
    │   │   │       # ⚠️歧义点: 不需要手动调用persist(), Chroma自动持久化
    │   │   │
    │   │   ├─ 【步骤7】重新初始化BM25检索器
    │   │   │   self._init_bm25_retriever()
    │   │   │   # ⚠️重要: 新增文档后，必须重新构建BM25索引
    │   │   │   # 原因: BM25索引是基于所有文档的，新增文档需要更新索引
    │   │   │
    │   │   ├─ 【步骤8】生成结果消息
    │   │   │   result_msg = (
    │   │   │       f"知识库更新完成! \n"
    │   │   │       f"成功处理: {success_count}个文件\n"
    │   │   │       f"失败: {fail_count}个文件\n"
    │   │   │       f"新增分块: {len(all_chunks)}个"
    │   │   │   )
    │   │   │   if fail_files:
    │   │   │       result_msg += f"\n失败文件: \n" +
"\n".join(fail_files)
    │   │   │   # 格式化消息，便于显示给用户
    │   │   │
```

```
    │   │   └─ 【返回】(all_chunks, result_msg)
    │   │       # 返回元组: (新增分块列表, 结果消息)
    │
    ↓
    ├── 【2.5】私有方法: _scan_knowledge_folder() ← 扫描知识库文件夹
    │   │
    │   ├─ 调用时机:
    │   │   └─ update_knowledge_base() 的步骤1
    │   │
    │   ├─ 作用:
    │   │   ├─ 遍历知识库文件夹及其子文件夹
    │   │   ├─ 收集所有支持格式的文件
    │   │   └─ 返回文件路径列表
    │   │
    │   ├─ 执行步骤:
    │   │   │
    │   │   ├─ 【步骤1】确保知识库文件夹存在
    │   │   │   ensure_directory_exists(KNOWLEDGE_BASE_DIR)
    │   │   │   # 该函数会创建文件夹(如果不存在)
    │   │   │
    │   │   ├─ 【步骤2】初始化文件列表
    │   │   │   target_files = []
    │   │   │
    │   │   ├─ 【步骤3】遍历文件夹
    │   │   │   for root, _, files in os.walk(KNOWLEDGE_BASE_DIR):
    │   │   │   # os.walk(): 递归遍历文件夹
    │   │   │   # 返回: (当前目录, 子目录列表, 文件列表)
    │   │   │   # root: 当前目录路径
    │   │   │   # _: 子目录列表(不需要, 用_占位)
    │   │   │   # files: 文件列表
    │   │   │
    │   │   │   for file in files:
    │   │   │       file_path = os.path.join(root, file)
    │   │   │       # 拼接完整路径: root + file
    │   │   │       # 例如: "D:/kb" + "file.pdf" = "D:/kb/file.pdf"
    │   │   │
    │   │   │       if is_supported_format(file_path,
SUPPORTED_FORMATS):
    │   │   │       # 检查文件格式是否支持
    │   │   │       # SUPPORTED_FORMATS = [".pdf", ".docx", ".md",
".txt"]
    │   │   │
    │   │   │           target_files.append(file_path)
    │   │   │           # 添加到文件列表
    │   │   │
    │   │   └─ 【返回】target_files
    │   │       # 返回所有支持的文件路径列表
```

```
    │
    ↓
    ├── 【2.6】私有方法: _filter_files_to_update() ← 筛选需要更新的文件
    │   │
    │   ├─ 调用时机:
    │   │   └─ update_knowledge_base() 的步骤3
    │   │
    │   ├─ 作用:
    │   │   ├─ 对比文件MD5值
    │   │   ├─ 找出新增或修改的文件
    │   │   └─ 返回待更新文件列表
    │   │
    │   ├─ 执行步骤:
    │   │   │
    │   │   ├─ 【步骤1】提取已有文档的MD5列表
    │   │   │   existing_md5s = list(set([
    │   │   │       meta.get("file_md5", "")
    │   │   │       for meta in self.all_docs["metadatas"]
    │   │   │   ]))
    │   │   │   # ⚠️ 歧义点: 为什么要用 set() 去重?
    │   │   │   # 原因: 同一个文件可能有多个分块, 每个分块都有相同的MD5
    │   │   │   # set() 会去除重复的MD5值
    │   │   │   # 例如: ["abc", "abc", "def"] → ["abc", "def"]
    │   │   │
    │   │   │   # meta.get("file_md5", "")
    │   │   │   # 如果 metadata 中没有 file_md5 字段, 返回空字符串
    │   │   │   # 避免 KeyError
    │   │   │
    │   │   ├─ 【步骤2】初始化待处理文件列表
    │   │   │   pending_files = []
    │   │   │
    │   │   ├─ 【步骤3】对比MD5, 筛选新文件或修改过的文件
    │   │   │   for file_path in target_files:
    │   │   │       file_md5 = get_file_md5(file_path)
    │   │   │       # 计算当前文件的MD5值
    │   │   │
    │   │   │       if file_md5 not in existing_md5s:
    │   │   │       # 如果MD5不在已有列表中, 说明是新文件或修改过的文件
    │   │   │
    │   │   │           pending_files.append(file_path)
    │   │   │           # 添加到待处理列表
    │   │   │
    │   │   └─ 【返回】pending_files
    │   │       # 返回需要更新的文件路径列表
    │
    ↓
    ├── 【2.7】私有方法: _process_files() ← 批量处理文件
```

```
    │   │
    │   ├─ 调用时机:
    │   │   └─ update_knowledge_base() 的步骤5
    │   │
    │   ├─ 作用:
    │   │   ├─ 逐个加载、分块文件
    │   │   ├─ 收集所有分块
    │   │   └─ 统计成功和失败数量
    │   │
    │   ├─ 执行步骤:
    │   │   │
    │   │   ├─ 【步骤1】初始化结果变量
    │   │   │   all_chunks = []      # 所有分块列表
    │   │   │   success_count = 0    # 成功处理的文件数
    │   │   │   fail_count = 0       # 失败的文件数
    │   │   │   fail_files = []      # 失败的文件列表（含错误信息）
    │   │   │
    │   │   ├─ 【步骤2】逐个处理文件
    │   │   │   for file_path in file_paths:
    │   │   │   # 遍历所有待处理文件
    │   │   │
    │   │   │   try:
    │   │   │   # 尝试处理文件
    │   │   │
    │   │   │   │   # 1. 加载文档
    │   │   │   │   docs =
self.document_loader.load_document(file_path)
    │   │   │   │   # load_document() 会根据文件类型选择对应的加载器
    │   │   │   │   # 返回: [Document对象1, Document对象2, ...]
    │   │   │   │
    │   │   │   │   # 2. 分块处理
    │   │   │   │   chunks =
self.document_chunker.split_documents(docs)
    │   │   │   │   # split_documents() 会将文档切分成500字的块
    │   │   │   │   # 返回: [Chunk对象1, Chunk对象2, ...]
    │   │   │   │
    │   │   │   │   # 3. 添加到总列表
    │   │   │   │   all_chunks.extend(chunks)
    │   │   │   │   # extend(): 将列表中的所有元素添加到另一个列表
    │   │   │   │
    │   │   │   │   # 4. 成功计数
    │   │   │   │   success_count += 1
    │   │   │   │
    │   │   │   except Exception as e:
    │   │   │   # 如果处理失败
    │   │   │
    │   │   │   │   # 1. 失败计数
```

```
    │   │   │   │   fail_count += 1
    │   │   │   │
    │   │   │   │   # 2. 记录失败信息
    │   │   │   │   fail_files.append(f"{get_file_name(file_path)}:
{str(e)}")
    │   │   │   │   # 格式："文件名：错误信息"
    │   │   │   │
    │   │   │   │   # 3. 继续处理下一个文件
    │   │   │   │   continue
    │   │   │   │   # ⚠️ 歧义点：使用 continue 而不是 break
    │   │   │   │   # continue：跳过当前文件，继续处理下一个
    │   │   │   │   # break：直接结束整个循环
    │   │   │   │   # 这里应该用 continue，因为一个文件失败不应该影响其他文件
    │   │   │
    │   │   └─ 【返回】(all_chunks, success_count, fail_count,
fail_files)
    │   │       # 返回元组：(所有分块, 成功数, 失败数, 失败文件列表)
    │
    ↓
    ├── 【2.8】公共方法：get_vector_db() ← 便捷方法：获取向量数据库对象
    │   │
    │   ├─ 调用时机：
    │   │   └─ 外部需要访问向量库时
    │   │
    │   ├─ 代码：
    │   │   def get_vector_db(self):
    │   │       return self.vector_db
    │   │
    │   ├─ 为什么需要这个方法？
    │   │   ├─ 封装：外部不需要知道 vector_db 是如何存储的
    │   │   ├─ 安全：可以在方法内部添加验证逻辑
    │   │   └─ 一致性：如果以后改变存储方式，只需修改这个方法
    │   │
    │   ├─ 使用示例：
    │   │   kb = KnowledgeBase(embeddings)
    │   │   vector_db, _ = kb.load_vector_db()
    │   │   # 直接访问（不推荐）
    │   │   # vector_db = kb.vector_db
    │   │
    │   │   # 通过方法访问（推荐）
    │   │   vector_db = kb.get_vector_db()
    │   │
    │   └─ 返回值：
    │       └─ Chroma 对象
    │
    ↓
    ├── 【2.9】公共方法：get_bm25_retriever() ← 便捷方法：获取BM25检索器
```

```
│    │
│    ├─ 调用时机:
│    │    └─ 外部需要访问BM25检索器时
│    │
│    ├─ 代码:
│    │    def get_bm25_retriever(self):
│    │        return self.bm25_retriever
│    │
│    ├─ 注意:
│    │    └─ 如果没有文档，返回 None
│    │        # 原因: BM25需要文档才能创建索引
│    │        # 首次运行时，向量库为空，BM25为None
│    │        # 调用者需要检查返回值
│    │
│    ├─ 使用示例:
│    │    kb = KnowledgeBase(embeddings)
│    │    vector_db, _ = kb.load_vector_db()
│    │
│    │    bm25 = kb.get_bm25_retriever()
│    │    if bm25 is not None:
│    │        # 使用BM25
│    │        pass
│    │    else:
│    │        # 不使用BM25
│    │        pass
│    │
│    └─ 返回值:
│         └─ BM25Retriever 对象或 None
│
↓
└── 【2.10】公共方法: get_stats() ← 便捷方法: 获取知识库统计信息
     │
     ├─ 调用时机:
     │    └─ 需要显示知识库统计时（如侧边栏显示文档数量）
     │
     ├─ 执行步骤:
     │    │
     │    ├─ 【步骤1】检查是否有文档
     │    │    if not self.all_docs:
     │    │        return {
     │    │            "document_count": 0,
     │    │            "chunk_count": 0,
     │    │            "unique_files": 0
     │    │        }
     │    │    # ⚠️ 歧义点: 检查的是 all_docs 本身是否为 None 或空
     │    │    # 而不是检查 all_docs["documents"] 是否为空
     │    │
```

```
        │   ├─ 【步骤2】计算唯一文件数
        │   │   unique_files = set([
        │   │       meta.get("file_name", "")
        │   │       for meta in self.all_docs["metadatas"]
        │   │   ])
        │   │   # ⚠️ 歧义点：为什么要用 file_name 而不是 file_md5?
        │   │   # file_name：文件名，用户更容易理解
        │   │   # file_md5：文件哈希，更准确但用户不直观
        │   │   # 这里用 file_name 是为了让用户看到"10个文件"
        │   │   # 而不是"10个MD5值"
        │   │
        │   ├─ 【步骤3】返回统计信息
        │   │   return {
        │   │       "document_count": len(unique_files),  # 唯一文件数
        │   │       "chunk_count": len(self.all_docs["documents"]),  #
分块总数
        │   │       "unique_files": len(unique_files)  # 唯一文件数（与
document_count相同）
        │   │   }
        │   │   # ⚠️ 歧义点：document_count 和 unique_files 是相同的
        │   │   # 为什么要重复?
        │   │   # 原因：为了兼容性和清晰度
        │   │   # document_count：文档数量（可能包含重复文件的不同版本）
        │   │   # unique_files：唯一文件数量
        │   │   # 在当前实现中，两者相同，但保留了不同字段以便未来扩展
        │
        └─ 返回值：
            └─ 字典：{"document_count": N, "chunk_count": M,
"unique_files": N}

【完成】KnowledgeBase 类定义完成
```

⚠️ 关键改动说明（新旧对比）

特性	旧版（函数式）	新版（类式）
封装方式	独立函数：init_embeddings(), init_vector_db() 等	封装在 KnowledgeBase 类中
向量库导入	langchain_community	langchain_chroma（新版）
更新方法名	build_or_update_knowledge_base	update_knowledge_base（简化）
数据结构	返回 documents, metadatas	

特性	旧版（函数式）	新版（类式）
		返回 documents, metadatas, ids（更完整）
便捷方法	无	get_vector_db(), get_bm25_retriever(), get_stats()
错误处理	基本处理	完善的错误处理和统计
私有方法	无	使用 _ 开头的私有方法
子组件	函数参数传入	在类初始化时创建

📋 实际调用方式

```
# ===== 新版方式（推荐）=====
from knowledge import KnowledgeBase

# 1. 创建知识库对象
kb = KnowledgeBase(embedding_model)
# ⚠️ 注意：这里只是创建对象，没有加载向量库

# 2. 加载或创建向量库
vector_db, all_docs = kb.load_vector_db()
# 首次运行：创建新的空向量库
# 后续运行：加载已有的向量库

# 3. 更新知识库（如果需要）
chunks, msg = kb.update_knowledge_base()
# 只处理新增或修改的文件
# 返回：(新增分块, 结果消息)

# 4. 获取组件（使用便捷方法）
vector_db = kb.get_vector_db()  # 获取向量库
bm25_retriever = kb.get_bm25_retriever()  # 获取BM25检索器

# 5. 获取统计信息（使用便捷方法）
stats = kb.get_stats()
print(f"文档数：{stats['document_count']}")
print(f"分块数：{stats['chunk_count']}")
print(f"唯一文件数：{stats['unique_files']}")
```

```python
# ===== 旧版方式（已废弃）=====
# from knowledge import init_embeddings, init_vector_db, build_or_update_knowledge_base
#
# # 1. 初始化嵌入模型
# embeddings = init_embeddings()
#
# # 2. 初始化向量库
# vector_db, all_docs = init_vector_db(embeddings)
#
# # 3. 更新知识库
# chunks, msg = build_or_update_knowledge_base(embeddings, vector_db, all_docs)
```

🔍 便捷方法/函数的作用

方法名	类型	作用	调用方式	返回值
`get_vector_db()`	类方法	获取向量数据库对象	`kb.get_vector_db()`	Chroma对象
`get_bm25_retriever()`	类方法	获取BM25检索器	`kb.get_bm25_retriever()`	BM25Retriever对象或None
`get_stats()`	类方法	获取统计信息	`kb.get_stats()`	字典（文档数、分块数、唯一文件数）
`_init_bm25_retriever()`	私有方法	初始化BM25（内部使用）	不建议直接调用	无
`_scan_knowledge_folder()`	私有方法	扫描文件夹（内部使用）	不建议直接调用	文件路径列表
`_filter_files_to_update()`	私有方法	筛选文件（内部使用）	不建议直接调用	待更新文件列表
`_process_files()`	私有方法	处理文件（内部使用）	不建议直接调用	(分块, 成功数, 失败数, 失败列表)

💡 私有方法 vs 公共方法

```python
# 公共方法（可以直接调用）
kb.load_vector_db()                # ✅ 推荐：加载向量库
kb.update_knowledge_base()         # ✅ 推荐：更新知识库
kb.get_vector_db()                 # ✅ 推荐：获取向量库
kb.get_bm25_retriever()            # ✅ 推荐：获取BM25
kb.get_stats()                     # ✅ 推荐：获取统计

# 私有方法（以下划线开头）
kb._init_bm25_retriever()          # ⚠️ 不推荐：内部方法，已在load_vector_db()中调用
kb._scan_knowledge_folder()        # ⚠️ 不推荐：内部方法，已在update_knowledge_base()中调用
kb._filter_files_to_update()       # ⚠️ 不推荐：内部方法，已在update_knowledge_base()中调用
kb._process_files()                # ⚠️ 不推荐：内部方法，已在update_knowledge_base()中调用

# 为什么不建议直接调用私有方法？
# 1. 封装性：私有方法可能依赖特定的上下文，直接调用可能出错
# 2. 维护性：私有方法可能在版本更新中改变
# 3. 安全性：私有方法没有参数验证，直接调用可能导致错误
```

⚠️ 歧义点详细说明

歧义1：self.all_docs 的结构

```python
# self.all_docs 是一个字典，包含三个字段
self.all_docs = {
    "documents": ["文档1内容", "文档2内容", ...],  # 文本内容列表
    "metadatas": [{"file_name": "a.pdf", ...}, ...],  # 元数据列表
    "ids": ["chunk_0", "chunk_1", ...]  # 唯一ID列表
}

# ⚠️ 歧义点：
# 1. 为什么不是列表而是字典？
#   原因：Chroma的get()方法返回字典，便于扩展
#   未来可能添加 "embeddings" 字段（存储嵌入向量）
#
# 2. documents 和 metadatas 是分开存储的
#   对应关系：通过索引对应
#   documents[0] 对应 metadatas[0] 对应 ids[0]
```

```python
#
# 3. 为什么要用 documents（复数）而不是 document?
#   原因：这是一个列表，包含多个文档
#   单数：document（一个文档）
#   复数：documents（多个文档）
```

歧义2：existing_md5s 使用 set() 去重

```python
# 为什么需要去重?
existing_md5s = list(set([
    meta.get("file_md5", "")
    for meta in self.all_docs["metadatas"]
]))

# 场景：同一个文件有多个分块
# 文件 "test.pdf" 有5个分块
# 每个分块的 metadata 都有相同的 file_md5 = "abc123"

# 不去重:
existing_md5s = ["abc123", "abc123", "abc123", "abc123", "abc123"]
# 结果：5个相同的MD5值

# 去重后:
existing_md5s = list(set(existing_md5s))  # ["abc123"]
# 结果：1个唯一的MD5值

# 为什么需要去重?
# 原因：对比MD5时，只需要知道"文件是否已存在"
# 不需要知道"文件有多少个分块"
# 如果不去重，会影响性能和准确性
```

歧义3：document_count vs unique_files

```
stats = kb.get_stats()
# 返回:
# {
#   "document_count": 10, # 文档数量
#   "chunk_count": 500,  # 分块数量
#   "unique_files": 10   # 唯一文件数量
# }
```

```python
# ⚠️ 歧义点：document_count 和 unique_files 为什么相同？

# 在当前实现中，两者相同：
unique_files = set([
    meta.get("file_name", "")
    for meta in self.all_docs["metadatas"]
])
return {
    "document_count": len(unique_files),   # len(unique_files) = 10
    "chunk_count": len(self.all_docs["documents"]),   # 总分块数
    "unique_files": len(unique_files)   # len(unique_files) = 10
}

# 为什么保留两个相同的字段？
# 原因：语义不同
# - document_count：文档数量（可能包含重复文件的不同版本）
# - unique_files：唯一文件数量（去重后的文件数）
#
# 未来可能的扩展：
# - 同一文件的不同版本都算作 document_count
# - 但 unique_files 去重后只算一个
#
# 例如：
# file1.pdf (v1) → 10个分块
# file1.pdf (v2) → 10个分块
# file2.pdf → 5个分块
#
# document_count = 2 (file1.pdf的v1和v2)
# unique_files = 1 (只有file1.pdf)
# chunk_count = 25 (10+10+5)
```

歧义4：为什么有私有方法？

```python
# 私有方法的命名约定：以下划线 _ 开头
def _init_bm25_retriever(self):
    pass

# Python中这不是强制的，只是一种约定
# 意图：告诉其他开发者，这是内部方法

# 为什么要区分私有和公共方法？
# 1. 封装性：外部不需要知道内部实现细节
# 2. 安全性：私有方法没有参数验证，直接调用可能出错
```

```python
# 3. 维护性：私有方法可能在版本更新中改变
# 4. 清晰度：公共方法提供简洁的接口，私有方法提供实现

# 示例：
# 公共方法：update_knowledge_base()
# - 提供简单的接口
# - 处理所有细节
# - 返回清晰的结果

# 私有方法：_scan_knowledge_folder(), _filter_files_to_update(), _process_files()
# - 提供具体的实现
# - 只在类内部调用
# - 不对外暴露

# 调用者只需要知道：
kb.update_knowledge_base()
# 不需要知道内部如何扫描、筛选、处理文件
```

4.8 retrieval/retriever.py 内部执行顺序

```
retrieval/retriever.py 被导入
    ↓
【第1步】导入依赖
    ├─ from config import (
    │   USE_MIXED_RETRIEVAL,
    │   USE_RERANKER,
    │   TOP_K_WIDE,
    │   VECTOR_WEIGHT,
    │   BM25_WEIGHT
    │   )
    ↓
【第2步】定义类 RetrieverBuilder（注意：实际代码使用类）
    ↓
【第2.1步】类初始化 __init__
    ├─ 接收参数：
    │   ├─ vector_db：向量数据库对象（必选）
    │   ├─ bm25_retriever：BM25检索器对象（可选，默认None）
    │   └─ reranker：重排序模型对象（可选，默认None）
    ├─ 保存实例变量：
    │   ├─ self.vector_db = vector_db
    │   ├─ self.bm25_retriever = bm25_retriever
    │   └─ self.reranker = reranker
```

```
    └─ 完成初始化
    ↓
【第2.2步】核心方法: build_retriever()  ← 构建最终检索器
    ↓
  【步骤1】创建基础向量检索器
        ├─ 调用 vector_db 的 as_retriever() 方法:
        │   vector_retriever = self.vector_db.as_retriever(
        │       search_kwargs={"k": TOP_K_WIDE}
        │   )
        └─ 说明:
            ├─ TOP_K_WIDE 从配置读取, 通常是10
            ├─ 表示先召回10个候选文档
            └─ 这是基础检索器, 后续可以在此基础上增强
    ↓
  【步骤2】构建混合检索器(向量 + BM25)
        ├─ 判断条件:
        │   if USE_MIXED_RETRIEVAL and self.bm25_retriever is not None:
        │   ├─ USE_MIXED_RETRIEVAL: 从配置读取, True表示启用混合检索
        │   └─ self.bm25_retriever is not None: BM25检索器已初始化(有文
档)
        │   └─ 两个条件都满足才启用混合检索
        │
        ├─ 【满足条件时】调用内部方法:
        │   base_retriever = self._build_ensemble_retriever(
        │       vector_retriever,
        │       self.bm25_retriever
        │   )
        │   ↓
        │   # 调用私有方法 _build_ensemble_retriever()
        │   ├─ 延迟导入(在方法内部导入):
        │   │   from langchain.retrievers import EnsembleRetriever
        │   │   # 为什么要延迟导入?
        │   │   # 1. 避免模块导入时的循环依赖
        │   │   # 2. 提高模块导入速度(只在需要时导入)
        │   │   # 3. 如果LangChain未安装, 不会影响模块定义
        │   │
        │   ├─ 创建集成检索器(混合检索):
        │   │   return EnsembleRetriever(
        │   │       retrievers=[vector_retriever, bm25_retriever],
        │   │       weights=[VECTOR_WEIGHT, BM25_WEIGHT]
        │   │   )
        │   │   # retrievers: 两个检索器的列表
        │   │   # weights: 权重列表, 对应上面的检索器
        │   │   #    VECTOR_WEIGHT=0.7(向量检索权重)
        │   │   #    BM25_WEIGHT=0.3(BM25检索权重)
        │   │   #    两者之和应该为1.0
        │   │
```

```
        │    └─ 返回混合检索器对象
        │    # 工作原理:
        │    # 1. 向量检索器返回10个文档及分数
        │    # 2. BM25检索器返回10个文档及分数
        │    # 3. 对分数进行加权合并:
        │    #    合并分数 = 向量分数×0.7 + BM25分数×0.3
        │    # 4. 按合并分数排序, 返回前10个文档
        │
        └─ 【不满足条件时】只使用向量检索:
             base_retriever = vector_retriever
             # 情况1: USE_MIXED_RETRIEVAL=False(配置禁用了混合检索)
             # 情况2: self.bm25_retriever is None(没有文档, BM25未初始化)
    ↓
【步骤3】构建重排序检索器
        ├─ 判断条件:
        │    if USE_RERANKER and self.reranker is not None:
        │    ├─ USE_RERANKER: 从配置读取, True表示启用重排序
        │    └─ self.reranker is not None: 重排序器已初始化
        │    └─ 两个条件都满足才启用重排序
        │
        ├─ 【满足条件时】调用内部方法:
        │    final_retriever =
self._build_rerank_retriever(base_retriever)
        │    ↓
        │    # 调用私有方法 _build_rerank_retriever()
        │    ├─ 延迟导入:
        │    │   from langchain.retrievers import
ContextualCompressionRetriever
        │    │   # 同样使用延迟导入避免循环依赖
        │    │
        │    ├─ 创建上下文压缩检索器(包含重排序功能):
        │    │   return ContextualCompressionRetriever(
        │    │       base_compressor=self.reranker,
        │    │       base_retriever=base_retriever
        │    │   )
        │    │   # base_compressor: 压缩器(这里用作重排序器)
        │    │   # base_retriever: 基础检索器(可能是向量检索或混合检索)
        │    │
        │    └─ 返回重排序检索器对象
        │    # 工作原理:
        │    # 1. 先调用 base_retriever 检索10个候选文档
        │    # 2. 将这10个文档和问题一起传给 reranker
        │    # 3. reranker 对每个文档评分(使用交叉编码器)
        │    # 4. 选择 Top-N(由reranker的top_n参数决定)
        │    # 5. 返回最终筛选出的文档
        │
        └─ 【不满足条件时】不使用重排序:
```

```
            final_retriever = base_retriever
            # 情况1: USE_RERANKER=False（配置禁用了重排序）
            # 情况2: self.reranker is None（重排序器未初始化）
    ↓
【返回】最终检索器对象
        return final_retriever
        # 返回的检索器类型:
        # - 最简单: VectorStoreRetriever（仅向量检索）
        # - 中等: EnsembleRetriever（向量+BM25混合检索）
        # - 最复杂: ContextualCompressionRetriever（向量+BM25+重排序）
    ↓
【第2.3步】私有方法: _build_ensemble_retriever()  ← 构建混合检索器（内部方法，
以_开头）
    ├─ 参数:
    │   ├─ vector_retriever: 向量检索器
    │   └─ bm25_retriever: BM25检索器
    ├─ 延迟导入:
    │   from langchain.retrievers import EnsembleRetriever
    └─ 创建并返回集成检索器:
        return EnsembleRetriever(
            retrievers=[vector_retriever, bm25_retriever],
            weights=[VECTOR_WEIGHT, BM25_WEIGHT]
        )
    ↓
【第2.4步】私有方法: _build_rerank_retriever()  ← 构建重排序检索器（内部方法，
以_开头）
    ├─ 参数:
    │   └─ base_retriever: 基础检索器（向量或混合检索）
    ├─ 延迟导入:
    │   from langchain.retrievers import ContextualCompressionRetriever
    └─ 创建并返回重排序检索器:
        return ContextualCompressionRetriever(
            base_compressor=self.reranker,
            base_retriever=base_retriever
        )
    ↓
【第2.5步】便捷函数: build_retriever()  ← 便捷函数（非类方法，顶层函数）
    ├─ 参数:
    │   ├─ vector_db: 向量数据库对象（必选）
    │   ├─ bm25_retriever: BM25检索器对象（可选，默认None）
    │   └─ reranker: 重排序模型对象（可选，默认None）
    ├─ 创建构建器对象:
    │   builder = RetrieverBuilder(vector_db, bm25_retriever, reranker)
    ├─ 调用构建方法:
    │   return builder.build_retriever()
    └─ 返回最终检索器对象
```

```
    # 便捷函数的作用:
    # 提供简洁的接口，隐藏类的使用细节
    # 用户可以直接调用 build_retriever()，而不需要先创建 RetrieverBuilder 实例
    # 这是"工厂模式"的一种体现
    ↓
【完成】类定义和便捷函数定义完成
```

⚠️ **关键改动说明（与旧版对比）：**

特性	旧版	新版（实际代码）
封装方式	独立函数	封装在 RetrieverBuilder 类中
导入方式	在模块顶部导入	延迟导入（在方法内部导入）
私有方法	无	使用 _ 开头的方法（内部使用）
便捷函数	build_final_retriever	build_retriever（名称简化）
灵活性	较低	较高（通过类可以扩展更多功能）

📋 **实际调用方式：**

```python
# 旧版方式（已废弃）
# from retrieval import build_final_retriever
# retriever = build_final_retriever(vector_db, bm25_retriever, reranker)

# 新版方式（实际使用）
from retrieval import build_retriever

# 方式1：使用便捷函数（推荐）
retriever = build_retriever(
    vector_db=vector_db,
    bm25_retriever=bm25_retriever,  # 可选
    reranker=reranker               # 可选
)

# 方式2：使用类（更灵活）
from retrieval import RetrieverBuilder

builder = RetrieverBuilder(
```

```python
        vector_db=vector_db,
        bm25_retriever=bm25_retriever,
        reranker=reranker
    )
    retriever = builder.build_retriever()
```

🔍 **便捷函数的作用：**

函数名	作用	调用方式	返回值
`build_retriever()`	构建最终检索器	`build_retriever(vector_db, bm25, reranker)`	检索器对象（可能是VectorStoreRetriever、EnsembleRetriever或ContextualCompressionRetriever）

💡 **延迟导入的优点：**

1. **避免循环依赖**：如果两个模块互相导入，延迟导入可以打破循环
2. **提高性能**：只在需要时才导入，减少启动时间
3. **提高健壮性**：如果LangChain未安装或版本不兼容，不会影响模块定义
4. **灵活性更高**：可以根据配置动态选择不同的实现

⚠️ **歧义点说明：**

歧义1：base_retriever 的含义

- 在步骤2中，`base_retriever` 可能是 `EnsembleRetriever`（混合检索）或 `VectorStoreRetriever`（向量检索）
- 在步骤3中，`base_retriever` 作为 `ContextualCompressionRetriever` 的基础
- **建议命名**：可以改为 `intermediate_retriever` 更清晰

歧义2：reranker 的角色

- `self.reranker` 存储的是重排序器对象（`CrossEncoderReranker`）
- 但在 `ContextualCompressionRetriever` 中，它作为 `base_compressor` 传入
- 这是因为 LangChain 的 `ContextualCompressionRetriever` 设计时将重排序器视为一种"压缩器"（从10个压缩到3个）
- **理解方式**：可以把重排序理解为"压缩"候选文档列表

歧义3：私有方法的可见性

- 方法名以 `_` 开头（如 `_build_ensemble_retriever`）
- 在 Python 中，这只是约定，不是强制
- 这些方法仍然可以被外部调用，但不推荐
- **理解方式**：类似于"受保护"的方法，应该只在类内部使用

4.9 retrieval/reranker.py 内部执行顺序

```
retrieval/reranker.py 被导入
    ↓
【第1步】导入依赖
    ├─ from langchain_community.cross_encoders import
HuggingFaceCrossEncoder
    ├─ from langchain_classic.retrievers.document_compressors import
CrossEncoderReranker
    └─ from config import RERANKER_MODEL, DEVICE, TOP_K_FINAL
    ↓
【第2步】定义函数
    ↓
函数: init_reranker()
    ├─ 创建交叉编码器模型:
    │   model = HuggingFaceCrossEncoder(
    │       model_name=RERANKER_MODEL,
    │       model_kwargs={"device": DEVICE}
    │   )
    ├─ 创建重排序器:
    │   reranker = CrossEncoderReranker(
    │       model=model,
    │       top_n=TOP_K_FINAL  # 从10个中选3个
    │   )
    └─ 返回 reranker
    ↓
【完成】函数定义完成
```

4.10 generation/prompts.py 内部执行顺序

```
generation/prompts.py 被导入
    ↓
【第1步】定义提示词模板（按使用顺序）
```

```
     ↓
模板1: SINGLE_TURN_PROMPT  ← 单轮问答
     └─ 定义完整的提示词字符串
   你是一个专业的文档问答助手，必须严格遵守以下规则:

        1. 仅基于【检索到的文档内容】回答用户的问题
        2. 如果没有相关信息，直接说"找不到"
        3. 回答要清晰、准确、分点说明
        4. 每句话结尾标注【来源: 文件名 第X页】

   【检索到的文档内容】:
        {context}

   【用户的问题】:
        {question}

   【你的回答】:
     ↓
模板2: MULTI_TURN_PROMPT  ← 多轮对话
     └─ 定义包含对话历史的提示词字符串
   你是一个专业的文档问答助手...

   【对话历史】:
        {chat_history}

   【检索到的文档内容】:
        {context}

   【用户的当前问题】:
        {question}

   【你的回答】:
     ↓
模板3: QUERY_REWRITE_PROMPT  ← 问题改写
     └─ 定义问题改写提示词字符串
   请结合【对话历史】，把用户的【当前问题】改写成一个完整、无指代的问题。

   要求:
        1. 补全指代（如"它""这个"）
        2. 保留原问题的核心意图
        3. 只输出改写后的问题

   【对话历史】:
        {chat_history}

   【用户的当前问题】:
        {question}
```

```
    【改写后的问题】:
    ↓
【完成】所有模板定义完成
```

4.11 generation/qa_chain.py 内部执行顺序

```
generation/qa_chain.py 被导入
    ↓
【第1步】导入依赖
    ├─ from langchain_community.llms import Ollama
    ├─ from generation.prompts import (
    │   SINGLE_TURN_PROMPT,
    │   MULTI_TURN_PROMPT,
    │   QUERY_REWRITE_PROMPT
    │   )
    └─ from config import USE_MULTI_TURN, LLM_TEMPERATURE
    ↓
【第2步】定义辅助函数
    ↓
函数1: init_llm()  ← 初始化大模型
    ├─ 创建Ollama实例:
    │   llm = Ollama(
    │       model=config.LLM_MODEL,
    │       temperature=LLM_TEMPERATURE
    │   )
    └─ 返回 llm
    ↓
函数2: format_chat_history(history)  ← 格式化对话历史
    ├─ formatted = []
    ├─ for msg in history:
    │   ├─ if msg["role"] == "user":
    │   │   └─ formatted.append(f"用户: {msg['content']}")
    │   └─ elif msg["role"] == "assistant":
    │       └─ formatted.append(f"助手: {msg['content']}")
    └─ 返回 "\n".join(formatted)
    ↓
函数3: format_source_docs(docs)  ← 格式化源文档
    ├─ formatted_docs = []
    ├─ for doc in docs:
    │   ├─ file_name = doc.metadata.get('file_name', '未知')
    │   ├─ page = doc.metadata.get('page', '未知')
    │   └─ formatted_docs.append(
    │       f"【来源: {file_name} 第{page}页】\n{doc.page_content}"
```

```
    │   )
    └─ 返回 "\n\n".join(formatted_docs)
    ↓
【第3步】定义类
    ↓
类: QAChain
    ↓
 方法1: __init__(self, llm, retriever)
        ├─ 接收LLM和检索器
        ├─ 保存到实例变量:
        │   self.llm = llm
        │   self.retriever = retriever
        └─ 完成
    ↓
 方法2: answer(self, question, chat_history)  ← 主要方法
        ↓
   步骤2.1: 问题改写（多轮对话时）
            └─ if USE_MULTI_TURN and len(chat_history) > 0:
                ├─ formatted_history =
format_chat_history(chat_history)
                ├─ prompt = QUERY_REWRITE_PROMPT.format(
                │   chat_history=formatted_history,
                │   question=question
                │   )
                ├─ rewritten_question = self.llm.invoke(prompt).strip()
                └─ search_query = rewritten_question
            └─ else:
                └─ search_query = question
        ↓
   步骤2.2: 检索文档
            └─ docs = self.retriever.invoke(search_query)
                ├─ 调用检索器链路
                ├─ 经过: 向量检索 → 混合检索 → 重排序
                └─ 返回Top-3 文档片段
        ↓
   步骤2.3: 格式化上下文
            └─ context = format_source_docs(docs)
                └─ 得到格式化的文档内容
        ↓
   步骤2.4: 构建提示词
            └─ if USE_MULTI_TURN and len(chat_history) > 0:
                ├─ prompt = MULTI_TURN_PROMPT.format(
                │   chat_history=format_chat_history(chat_history),
                │   context=context,
                │   question=question
                │   )
                └─ else:
```

```
                        ├─ prompt = SINGLE_TURN_PROMPT.format(
                        │   context=context,
                        │   question=question
                        │   )
          ↓
    步骤2.5: 生成答案
              └─ answer = self.llm.invoke(prompt).strip()
                  ├─ 发送给LLM
                  ├─ LLM推理生成答案
                  └─ 返回生成的文本
          ↓
    步骤2.6: 返回结果
              └─ return {
                  "answer": answer,
                  "source_docs": docs,
                  "rewritten_question": rewritten_question if
rewritten_question != question else None
              }
     ↓
【完成】类定义完成
```

---

5. 完整调用关系图

5.1 离线阶段调用关系图

```
main.py
    ↓
┌──────────────────────────────────────────────────────────────┐
│ init_system_components()                                      │
├──────────────────────────────────────────────────────────────┤
│                                                               │
│  ┌──────────────────────────────────────────────────┐        │
│  │ init_embeddings()                                 │        │
│  │   knowledge/knowledge_base.py                     │        │
│  │   → HuggingFaceEmbeddings                        │        │
│  │   → 返回 embeddings                              │        │
│  └──────────────────────────────────────────────────┘        │
│                            ↓                                  │
│  ┌──────────────────────────────────────────────────┐        │
│  │ init_llm()                                        │        │
│  │   generation/qa_chain.py                         │        │
│  │   → Ollama()                                      │        │
```

```
│   │   → 返回 llm                                    │     │
│   └──────────────────────────────────────────────────┘     │
│                           ↓                                 │
│   ┌──────────────────────────────────────────────────┐     │
│   │ init_vector_db(embeddings)                       │     │
│   │   knowledge/knowledge_base.py                    │     │
│   │   ↓                                              │     │
│   │   if 首次运行:                                    │     │
│   │     → Chroma() 创建新库                           │     │
│   │   else:                                          │     │
│   │     → Chroma() 加载已有库                         │     │
│   │   → 返回 vector_db, all_docs                      │     │
│   └──────────────────────────────────────────────────┘     │
│                           ↓                                 │
│   ┌──────────────────────────────────────────────────┐     │
│   │ init_bm25_retriever(all_docs)                    │     │
│   │   knowledge/knowledge_base.py                    │     │
│   │   ↓                                              │     │
│   │   if all_docs为空:                                │     │
│   │     → 返回 None                                   │     │
│   │   else:                                          │     │
│   │     → BM25Retriever.from_documents()             │     │
│   │     → 返回 bm25_retriever                         │     │
│   └──────────────────────────────────────────────────┘     │
│                           ↓                                 │
│   ┌──────────────────────────────────────────────────┐     │
│   │ init_reranker()                                  │     │
│   │   retrieval/reranker.py                          │     │
│   │   → HuggingFaceCrossEncoder                      │     │
│   │   → CrossEncoderReranker                         │     │
│   │   → 返回 reranker                                 │     │
│   └──────────────────────────────────────────────────┘     │
│                           ↓                                 │
│   ┌──────────────────────────────────────────────────┐     │
│   │ build_final_retriever(...)                       │     │
│   │   retrieval/retriever.py                         │     │
│   │   ↓                                              │     │
│   │   1. vector_db.as_retriever()                    │     │
│   │   2. if 混合检索:                                 │     │
│   │        → EnsembleRetriever()                     │     │
│   │   3. if 重排序:                                   │     │
│   │        → ContextualCompressionRetriever()        │     │
│   │   → 返回 final_retriever                          │     │
│   └──────────────────────────────────────────────────┘     │
│                           ↓                                 │
│   ┌──────────────────────────────────────────────────┐     │
│   │ QAChain(llm, final_retriever)                    │     │
```

```
│   │   generation/qa_chain.py                          │      │
│   │   → 返回 qa_chain                                 │      │
│   └──────────────────────────────────────────────────┘      │
│                                                              │
└──────────────────────────────────────────────────────────────┘

    ↓
返回 components 字典
```

5.2 更新知识库调用关系图

```
main.py
    ↓
┌──────────────────────────────────────────────────────────────┐
│ build_or_update_knowledge_base(embeddings, vector_db,        │
│                                all_docs)                     │
├──────────────────────────────────────────────────────────────┤
│   knowledge/knowledge_base.py                                │
└──────────────────────────────────────────────────────────────┘

    ↓
┌──────────────────────────────────────────────────────────────┐
│ 步骤1: 扫描知识库文件夹                                       │
│ → os.walk(KNOWLEDGE_BASE_DIR)                                │
│ → 收集所有支持的文件                                          │
└──────────────────────────────────────────────────────────────┘

    ↓
┌──────────────────────────────────────────────────────────────┐
│ 步骤2: 筛选需要更新的文件                                     │
│ → get_file_md5(file_path)                                    │
│   utils/file_utils.py                                        │
│   → 对比已有文档的MD5                                         │
│ → 得到 pending_files 列表                                     │
└──────────────────────────────────────────────────────────────┘

    ↓
对于每个文件 in pending_files:
    ↓
    ┌──────────────────────────────────────────────────┐
    │ DocumentLoader.load_document(file_path)          │
    │   knowledge/document_loader.py                   │
    ├──────────────────────────────────────────────────┤
    │ 1. _is_supported(file_path)                      │
    │    → get_file_extension()                        │
    │    → 检查是否支持                                 │
    │                                                  │
    │ 2. _get_loader(file_path)                        │
```

```
│     → 根据扩展名选择加载器                          │
│     → PyPDFLoader / Docx2txtLoader / ...          │
│                                                   │
│ 3. loader.load()                                  │
│     → 加载文档内容                                  │
│                                                   │
│ 4. _add_metadata(documents, file_path)            │
│     → get_file_md5()                              │
│     → get_file_name()                             │
│     → 添加元数据                                    │
│                                                   │
│ → 返回 documents                                   │
└───────────────────────────────────────────────────┘
    ↓
    ┌─────────────────────────────────────────────┐
    │ DocumentChunker.split_documents(documents)  │
    │   knowledge/chunker.py                      │
    ├─────────────────────────────────────────────┤
    │ if CHUNK_STRATEGY == "markdown":            │
    │   → MarkdownHeaderTextSplitter              │
    │   → RecursiveCharacterTextSplitter（二次）   │
    │ else:                                       │
    │   → RecursiveCharacterTextSplitter          │
    │ → 返回 chunks                                │
    └─────────────────────────────────────────────┘
        ↓
扩展到 all_chunks 列表
↓
┌───────────────────────────────────────────────────┐
│ vector_db.add_documents(all_chunks)               │
│   Chroma                                          │
├───────────────────────────────────────────────────┤
│ 1. 计算每个分块的嵌入向量                            │
│     → embeddings.embed_documents()                │
│                                                   │
│ 2. 存入向量库                                       │
│     → 存储向量、元数据、内容                          │
│                                                   │
│ 3. 持久化到磁盘                                     │
│     → chroma_db.persist()                         │
└───────────────────────────────────────────────────┘
↓
┌───────────────────────────────────────────────────┐
│ init_bm25_retriever(all_docs)                     │
│   knowledge/knowledge_base.py                     │
│ → BM25Retriever.from_documents()                  │
│ → 返回新的 bm25_retriever                           │
```

```
    └──────────────────────────────────────────────────┘
    ↓
    ┌──────────────────────────────────────────────────┐
    │ build_final_retriever(...)                       │
    │   retrieval/retriever.py                         │
    │ → 重新构建检索器（包含新的BM25）                  │
    └──────────────────────────────────────────────────┘
    ↓
返回 (all_chunks, msg)
```

5.3 在线问答调用关系图

```
main.py
    ↓
┌──────────────────────────────────────────────────────────┐
│ qa_chain.answer(question, chat_history)                  │
├──────────────────────────────────────────────────────────┤
│   generation/qa_chain.py                                 │
└──────────────────────────────────────────────────────────┘
    ↓
    ┌──────────────────────────────────────────────────┐
    │ if 需要问题改写:                                  │
    │                                                  │
    │   format_chat_history(chat_history)              │
    │   → 格式化对话历史                                │
    │                                                  │
    │   QUERY_REWRITE_PROMPT.format(...)               │
    │   generation/prompts.py                          │
    │   → 构建改写提示词                                │
    │                                                  │
    │   llm.invoke(prompt)                             │
    │   → LLM改写问题                                   │
    │   → "它" → "RAG"                                 │
    │   → 返回 rewritten_question                      │
    │                                                  │
    │   search_query = rewritten_question              │
    │                                                  │
    │ else:                                            │
    │   search_query = question                        │
    └──────────────────────────────────────────────────┘
        ↓
        ┌──────────────────────────────────────────────┐
        │ final_retriever.invoke(search_query)         │
        │   retrieval/retriever.py                     │
```

```
├──────────────────────────────────────────────────────┤
│ 步骤1: 向量检索                                      │
│ → vector_retriever.invoke()                          │
│ → embeddings.embed_query()                           │
│ → Chroma.similarity_search()                         │
│ → 返回10个文档片段                                   │
│                                                      │
│ 步骤2: 混合检索（如果启用）                          │
│ → EnsembleRetriever                                  │
│ → 合并向量检索和BM25结果                             │
│ → 权重: 0.7向量 + 0.3BM25                            │
│                                                      │
│ 步骤3: 重排序（如果启用）                            │
│ → reranker.compress_documents()                      │
│ → CrossEncoderReranker                               │
│ → 计算相关性分数                                     │
│ → 选择Top-3                                          │
│                                                      │
│ → 返回 docs（3个文档片段）                           │
└──────────────────────────────────────────────────────┘

    ↓

    ┌──────────────────────────────────────────────┐
    │ format_source_docs(docs)                     │
    │   generation/qa_chain.py                     │
    │ → 格式化文档                                 │
    │ → 【来源: file1.pdf 第3页】                  │
    │    内容...                                   │
    └──────────────────────────────────────────────┘

        ↓

        ┌──────────────────────────────────────────┐
        │ 构建提示词                               │
        │   generation/prompts.py                  │
        ├──────────────────────────────────────────┤
        │ if 多轮对话:                             │
        │   → MULTI_TURN_PROMPT.format(...)        │
        │     - chat_history                       │
        │     - context                            │
        │     - question                           │
        │ else:                                    │
        │   → SINGLE_TURN_PROMPT.format(...)       │
        │     - context                            │
        │     - question                           │
        │ → 返回 prompt                            │
        └──────────────────────────────────────────┘

            ↓

            ┌──────────────────────────────────────┐
            │ llm.invoke(prompt)                   │
```

```
                        │   Ollama                                  │
                        ├──────────────────────────────────────────┤
                        │ 1. 接收提示词                              │
                        │ 2. 模型推理                                │
                        │ 3. 生成答案                                │
                        │ → 返回 answer                              │
                        └──────────────────────────────────────────┘

                            ↓
                            ┌──────────────────────────────────────┐
                            │ return {                              │
                            │   "answer": answer,                   │
                            │   "source_docs": docs,                │
                            │   "rewritten_question": ...           │
                            │ }                                     │
                            └──────────────────────────────────────┘

    ↓
main.py 处理返回结果
    ↓
显示答案和源文档
```

---

6. 数据流向图

6.1 离线阶段数据流

```
【数据源: 文档文件】
file1.pdf, file2.docx, file3.md, ...
    ↓
【加载阶段】
DocumentLoader.load_document()
    ↓
原始文档数据:
Document objects
  - page_content: "文档内容..."
  - metadata: {
      file_name: "file1.pdf",
      page: 1,
      ...
    }
    ↓
【分块阶段】
DocumentChunker.split_documents()
    ↓
```

```
分块数据:
Chunk objects
  - page_content: "分块内容（500字）..."
  - metadata: {
      file_name: "file1.pdf",
      page: 1,
      chunk_id: 0,
      ...
    }
    ↓
【向量化阶段】
embeddings.embed_documents(chunks)
    ↓
嵌入向量:
Vector objects
  - vector: [0.123, 0.456, ...] (768维)
  - metadata: {...}
  - content: "分块内容..."
    ↓
【存储阶段】
Chroma vector_db.add_documents()
    ↓
持久化存储:
磁盘数据库（CHROMA_DB_PATH）
  - collection
    - documents: [...]
    - embeddings: [[...], [...], ...]
    - metadatas: [{...}, {...}, ...]
    - ids: ["chunk_0", "chunk_1", ...]
```

6.2 在线阶段数据流

```
【用户输入】
用户问题: "什么是RAG？"
    ↓
【问题改写（多轮时）】
原始问题: "它" → 改写问题: "RAG"
    ↓
【检索阶段1: 向量检索】
问题: "什么是RAG？"
    ↓
问题向量化:
embeddings.embed_query()
    ↓
```

```
向量: [0.789, 0.234, ...]
    ↓
相似度搜索:
Chroma.similarity_search()
    ↓
返回10个候选文档
    ↓
【检索阶段2: 混合检索（可选）】
向量结果 + BM25结果
    ↓
融合排序:
EnsembleRetriever
    ↓
返回10个候选文档
    ↓
【检索阶段3: 重排序（可选）】
10个候选文档 + 问题
    ↓
重排序评分:
CrossEncoderReranker
    ↓
分数:
 文档1: 9.5分
 文档2: 8.8分
 文档3: 8.2分
  ...
    ↓
选择Top-3
    ↓
返回3个最终文档
    ↓
【格式化阶段】
文档对象 → 格式化文本
    ↓
格式化上下文:
【来源: file1.pdf 第3页】
RAG（Retrieval-Augmented Generation）是...

【来源: file2.md 第1页】
检索增强生成技术结合了...
    ↓
【提示词构建】
上下文 + 问题 + 提示词模板
    ↓
完整提示词:
你是一个专业的文档问答助手...
```

```
【检索到的文档内容】:
【来源: file1.pdf 第3页】
...

【用户的问题】:
什么是RAG?

【你的回答】:
    ↓
【答案生成】
提示词 → LLM推理
    ↓
答案:
RAG(Retrieval-Augmented Generation)是一种
结合了信息检索和文本生成的AI技术...
    ↓
【结果返回】
{
  "answer": "...",
  "source_docs": [...],
  "rewritten_question": null
}
    ↓
【显示】
用户界面显示答案和源文档
```

---

7. 执行时间线总结

7.1 首次运行(无向量库)

```
时间  操作          文件          耗时估计
──────────────────────────────────────────────────────────────────────
T0     streamlit run main.py   main.py                  -
T0.1   加载配置       config.py                 <1ms
T0.2   初始化嵌入模型    knowledge_base.py         5-10s(首次下载模型)
T0.3   初始化LLM               qa_chain.py               2-3s(首次下载模型)
T0.4   创建向量库      knowledge_base.py         <100ms
T0.5   初始化重排序器    reranker.py              3-5s(首次下载模型)
T0.6   构建检索器      retriever.py              <10ms
T0.7   创建问答链      qa_chain.py              <10ms
T0.8   显示Web界面     main.py                  <100ms
```

```
T0.9    等待用户操作              -                          -

──────────────────────────────────────────────────────────────────────
小计：约10-20秒（包含模型下载）
```

7.2 更新知识库

```
时间  操作          文件          耗时估计
──────────────────────────────────────────────────────────────────────
T1      点击更新按钮     main.py                  -
T1.1    扫描文件夹      knowledge_base.py        <1s
T1.2    筛选文件       knowledge_base.py        <1s
T1.3    加载文档       document_loader.py       1-5s/文档
T1.4    分块处理       chunker.py               1-3s/文档
T1.5    向量化        Chroma                   2-5s/文档
T1.6    存储向量库      Chroma                   1-3s/文档
T1.7    初始化BM25               knowledge_base.py        1-5s
T1.8    重建检索器      retriever.py             <10ms
T1.9    刷新页面       main.py                  <100ms

──────────────────────────────────────────────────────────────────────
小计：取决于文档数量，通常10-60秒
```

7.3 在线问答

```
时间  操作          文件          耗时估计
──────────────────────────────────────────────────────────────────────
T2      用户提问       main.py                  -
T2.1    问题改写（多轮）    qa_chain.py              1-3s
T2.2    向量检索       retriever.py             100-500ms
T2.3    混合检索（可选）    retriever.py             200-800ms
T2.4    重排序（可选）     retriever.py             500ms-2s
T2.5    格式化上下文      qa_chain.py              <100ms
T2.6    构建提示词       qa_chain.py              <50ms
T2.7    生成答案       qa_chain.py              3-10s（取决于模型）
T2.8    显示结果       main.py                  <100ms

──────────────────────────────────────────────────────────────────────
小计：通常5-15秒
```

8. 关键执行路径总结

8.1 最短路径（最小化功能）

```
配置: USE_MIXED_RETRIEVAL=False, USE_RERANKER=False

main.py
  → init_embeddings()
  → init_llm()
  → init_vector_db()
  → build_final_retriever()  ← 只有向量检索
  → QAChain()
  → 等待用户输入
  → qa_chain.answer()
    → retriever.invoke()    ← 只向量检索
    → llm.invoke()
```

8.2 标准路径（推荐配置）

```
配置: USE_MIXED_RETRIEVAL=True, USE_RERANKER=True

main.py
  → init_embeddings()
  → init_llm()
  → init_vector_db()
  → init_bm25_retriever()
  → init_reranker()
  → build_final_retriever()  ← 向量+BM25+重排序
  → QAChain()
  → 等待用户输入
  → qa_chain.answer()
    → 问题改写（多轮）
    → retriever.invoke()    ← 完整检索链路
    → llm.invoke()
```

8.3 最长路径（所有功能）

```
配置：所有开关都打开

main.py
  → init_embeddings()
  → init_llm()
  → init_vector_db()
  → init_bm25_retriever()
  → init_reranker()
  → build_final_retriever()
  → QAChain()
  → 等待用户输入
  → qa_chain.answer()
    → 问题改写（每次都执行）
    → retriever.invoke()
      → 向量检索
      → 混合检索
      → 重排序
    → 上下文压缩（如果启用）
    → 反馈循环（如果启用）
    → llm.invoke()
    → 多轮对话优化
```

---

9. 学习建议

9.1 推荐学习顺序

1. **第一阶段**：理解离线流程
   - 从 config.py 开始
   - 理解知识库构建流程
   - 重点看 knowledge_base.py
2. **第二阶段**：理解在线流程
   - 从用户提问开始
   - 理解检索链路
   - 重点看 retriever.py 和 qa_chain.py
3. **第三阶段**：理解数据流
   - 追踪文档如何变成向量
   - 追踪问题如何变成答案

- 重点看数据流向图

4. **第四阶段**：动手实践
   - 修改配置，观察变化
   - 关闭某些功能，观察影响
   - 添加新功能，练习调用

9.2 调试技巧

1. **打印执行顺序**

```python
print("[1] 加载配置...")
print("[2] 初始化模型...")
print("[3] 构建检索器...")
```

2. **打印数据流**

```python
print(f"问题: {question}")
print(f"检索到文档数: {len(docs)}")
print(f"生成的答案: {answer}")
```

3. **使用调试器**
   - 在关键位置设置断点
   - 逐步执行，观察变量变化
   - 理解调用栈

记住：理解执行顺序是掌握复杂系统的关键。按照这个指南，你可以清楚地知道每个函数什么时候被调用，数据如何流动，系统如何工作！